HLD Level 1.1

Requirements & Capacity Estimation - C++ / Systems Interview Notes

Goal: before choosing databases, queues, threads, protocols, or caches, determine what the system must do and what scale it must handle. Good HLD starts with constraints, not boxes.

1. Why requirements come first

The same feature can require completely different architectures depending on its non-functional requirements. A camera system allowed 2 seconds per frame is different from one requiring fresh results within 50 ms. A telemetry service for 100 devices is different from one serving 10 million devices.

Interview rule: do not immediately draw an architecture. First clarify requirements and scale.

2. Functional requirements

Functional requirements describe **what the system does** from the user's or another system's perspective.

Example - Device Telemetry System:
- Devices send telemetry.
- The system stores telemetry.
- Users query historical telemetry.
- The system can generate alerts for abnormal readings.

Good interview question: What are the 2-4 core features we need to support, and what is explicitly out of scope?

3. Non-functional requirements (NFRs)

NFRs describe **how well** the system must perform. These requirements usually drive the architecture more strongly than the feature list.

NFR	Meaning	Typical interview question
Latency	Time to complete one operation	Must an alert arrive within 100 ms or 10 s?
Throughput	Work handled per unit time	How many events/requests per second?
Availability	How often service is usable	Can the service tolerate downtime?
Durability	Accepted data survives failures	Can acknowledged telemetry be lost?
Consistency	What readers observe after writes	Must users immediately see newest value?
Scalability	Ability to handle growth	What happens at 10x traffic?

Latency vs throughput

Latency asks: how long does one operation take?
Throughput asks: how many operations can the system complete per second?

Example: one inference may take 40 ms latency, while parallel workers may collectively process many more than 25 requests/sec. Improving throughput does not automatically improve single-request latency.

4. Availability, reliability and durability

Availability: is the service reachable and able to respond?
Reliability: does the system behave correctly over time and under failures?
Durability: once data is acknowledged as stored, does it survive crashes or hardware failures?

Do not use these terms interchangeably. A service can temporarily be unavailable while its stored data remains durable.

5. Consistency

Strong consistency: after a successful write, subsequent reads should observe the latest committed value.
Eventual consistency: replicas may temporarily disagree, but converge later.

Ask whether stale data is acceptable. Device dashboards may tolerate a small delay; an order-matching engine generally cannot casually reorder or expose inconsistent order state.

6. Capacity estimation

Capacity estimation gives the architecture an approximate size. Interview estimates do not need perfect numbers. They need correct dimensions and reasonable assumptions.

Core quantities

RPS/QPS: requests or queries per second.
Concurrent connections: simultaneous active/open clients.
Bandwidth: bytes transferred per second.
Storage: data retained over time.

Useful formulas

```
Average RPS = requests per day / 86,400
Bandwidth = events/sec x bytes/event
Daily storage = events/sec x bytes/event x 86,400
Retention storage = daily storage x retention days x replication factor
```

Peak matters more than average. If average traffic is 10,000 RPS but traffic can spike 5x, discuss roughly 50,000 RPS peak rather than pretending load is uniform.

7. Worked example - telemetry ingestion

Assume:
- 100,000 devices
- each sends 1 message every 5 seconds
- each message is approximately 500 bytes
- retain raw data for 30 days
- use 3 copies of stored data for this rough estimate

Events/sec = 100,000 / 5 = **20,000 events/sec**
Ingress payload bandwidth = 20,000 x 500 = **10 MB/sec**
Raw storage/day = 10 MB/sec x 86,400 ~= **864 GB/day**
30-day raw storage ~= **25.9 TB**
With replication factor 3 ~= **77.8 TB** before indexes, metadata, compression and protocol overhead.

The estimate tells us to consider horizontally scalable ingestion, partitioning, buffering, storage lifecycle, compression and failure handling rather than a single-process toy design.

8. C++ / systems-specific estimation

For C++ systems interviews, also estimate resources inside a process. This is where generic HLD often becomes systems engineering.

```
Queue memory: queue_capacity x average_item_size
Buffer memory: concurrent_buffers x bytes_per_buffer
Thread cost: worker count, stack size, context switching and contention
Connection cost: socket state + application buffers + per-client metadata
CPU budget: work per item x items/sec
```

Example - bounded camera queue

Suppose capture produces 30 frames/sec while inference sustainably consumes 20 frames/sec. The backlog grows by 10 frames every second. An unbounded queue therefore increases memory use and, more importantly for a real-time pipeline, makes frames progressively stale.

That leads to architectural questions: Should the producer block? Should stale frames be dropped? Should we scale consumers? Can we batch? What latency is acceptable? The requirement determines the answer.

9. How to start an HLD interview

1. Clarify the core functional requirements.
2. Clarify latency, availability, durability and consistency expectations.
3. Ask for or assume scale: clients/devices, traffic, payload size and retention.
4. Make a few rough calculations.
5. State the important consequences of those numbers.
6. Only then draw the high-level architecture.

Example interviewer dialogue

Interviewer: Design a telemetry system for IoT devices.

Candidate: Before designing it, I'd like to clarify the requirements. Do devices only upload telemetry, or do we also need querying and real-time alerts? Approximately how many devices are active, how frequently do they send data, and what latency and retention do we require?

10. Common mistakes

Starting with technology: choosing Kafka, Redis or a database before establishing why.

Ignoring peak traffic: sizing only for average load.

Fake precision: calculating 12.873 TB instead of using useful approximations.

Confusing latency and throughput: more workers may improve throughput while contention worsens latency.

Ignoring payload size: 20,000 tiny events/sec and 20,000 video frames/sec are radically different systems.

Ignoring resource bounds: unlimited queues, threads, retries or connections eventually become failure mechanisms.

11. Interview checklist

Question	What you are trying to learn
What must the system do?	Functional scope
What is out of scope?	Prevent unnecessary design
How many users/devices?	System scale
Requests/events per second?	Throughput
Payload size?	Network + storage cost
Peak/average ratio?	Burst capacity
Latency target?	Processing choices
Availability target?	Redundancy/failover
Can data be lost?	Durability
Can reads be stale?	Consistency
Retention period?	Storage capacity/lifecycle

12. 60-second revision sheet

Functional requirements = what the system does.
Non-functional requirements = how well it must do it.
Latency = time per operation.
Throughput = operations per unit time.
Availability = service is usable.
Durability = accepted data survives failure.
Consistency = rules governing what readers observe after writes.
RPS = requests/day / 86,400.
Bandwidth = events/sec x bytes/event.
Storage/day = bandwidth x 86,400.
Always consider peak traffic and bounded resources.

13. Practice questions

1. A service receives 5 million requests/day. What is average RPS? What peak assumption would you discuss?
2. 500,000 devices send a 1 KB message every 10 seconds. Estimate events/sec and raw daily storage.
3. A producer creates 50,000 messages/sec and consumers process 40,000/sec. What happens over time?

4. For a live camera system, which matters more: processing every frame or keeping latency/freshness bounded? What requirement decides this?
5. Give one example where eventual consistency is acceptable and one where you would push for stronger consistency.

Ready for Level 1.2 when you can: clarify a vague system-design question, identify the important NFRs, perform rough traffic/storage calculations, and explain how those constraints influence the architecture.